Informe Técnico - Sokoban

Proceso de Desarrollo de Software (PDS) - Trabajo Integrador

Docente: Prof. Jonathan Pepe - Fundación UADE

Fecha de entrega: Noviembre 2026

Equipo de desarrollo: Abdo Ferez, Federico entrega individual (1 integrante).

Apellido y Nombre	Legajo	Mail
Abdo Ferez, Federico	1194765	fabdoferez@uade.edu.ar

Índice

1. Introducción
2. Descripción del juego
3. Arquitectura general y MVC
4. Patrones de diseño GoF aplicados
5. Patrones GRASP aplicados
6. Principios SOLID
7. Diseño de niveles
8. Interfaz gráfica (Swing)
9. Suite de pruebas
10. Métricas del proyecto
11. Decisiones de diseño relevantes
12. Uso de herramientas de IA
13. Conclusiones
14. Anexo - Diagramas de clases

1. Introducción

El presente informe documenta el diseño, la implementación y las decisiones técnicas del trabajo integrador de la materia Proceso de Desarrollo de Software: un juego Sokoban desarrollado en Java 21 con interfaz gráfica en Swing.

El objetivo del trabajo es aplicar de forma práctica los patrones de diseño estudiados en la materia (GoF, GRASP) y los principios SOLID, demostrando que cada patrón responde a un problema concreto del dominio y no a una imposición teórica.

El proyecto consta de 89 clases de producción y 22 clases de prueba, con una suite de 108 tests JUnit 5 que pasan en su totalidad (BUILD SUCCESS). El juego incluye 5 niveles con dificultad creciente, 21 sprites PNG, 9 efectos de sonido WAV, niebla de guerra, sistema de puntuación con estrellas y persistencia de progreso.

2. Descripción del juego

2.1 Reglas básicas

Sokoban es un juego de lógica donde el jugador debe empujar cajas hasta sus casillas de destino. El jugador solo empuja (nunca tira), no puede empujar más de una caja a la vez, ni atravesar paredes ni cajas.

2.2 Tipos de cajas

Tipo de caja	Símbolo	Comportamiento
Normal	$	Se empuja sin límite
Frágil	1-9	Soporta N empujes; el siguiente la rompe (derrota)
Llave	K	Abre muros al caer sobre el casillero cerrojo correspondiente; solo puede guardarse en su destino después de haber abierto su muro

2.3 Tipos de baldosas

Baldosa	Símbolo	Efecto
Piso		Sin efecto
Pared	#	Bloquea jugador y cajas, no deja ver
Vacío	%	Fuera del tablero, bloquea todo
Hielo		Desliza al ocupante hasta la siguiente baldosa que no sea hielo
Portal	P-S	Teletransporta al ocupante al portal pareado
Destino		Casilla donde guardar cajas (normal, frágil, llave). Vacío bloquea al jugador (es un "agujero"); lleno hace de puente
Casillero cerrojo	a-e	Al recibir la caja llave, abre todos los muros de la misma letra
Muro cerrado	A-E	Bloquea como pared hasta que la llave del grupo lo abre

2.4 Sistema de deshacer

Se permite deshacer los últimos 15 movimientos. Cada uso del botón restaura el estado del nivel 5 movimientos hacia atrás, con un máximo de 3 usos consecutivos. Al volver a mover, el contador de usos se reinicia.

2.5 Condiciones de victoria y derrota

- **Victoria:** todas las casillas de destino tienen una caja del tipo correcto guardada.
- **Derrota por caja rota (BROKEN):** una caja frágil recibe un empuje de más.
- **Derrota por guardado incorrecto (WRONG_STORAGE):** una caja queda guardada en un destino del tipo equivocado.
- **Derrota por llave prematura (EARLY_KEY_STORAGE):** una caja llave se guarda sin haber abierto su muro.
- **Derrota por deadlock (DEADLOCK):** la caja empujada queda trabada en una esquina o un bloque 2 × 2 irrecuperable. Tiene arreglo con deshacer.
- **Derrota por tiempo (TIME_OUT):** se agota el tiempo límite del nivel.

2.6 Puntaje y estrellas

- +1 punto por segundo restante
- +100 puntos por cada caja guardada
- +100 puntos por cada muro abierto
- -1 punto por cada empuje
- -250 puntos por cada uso del botón deshacer
- **Estrellas:** ≤ 50% del objetivo → 1, ≤ 85% → 2, > 85% → 3

3. Arquitectura general y MVC

3.1 Estructura de paquetes

```
src/main/java/com/sokoban/
  Main.java                  # punto de entrada (arma las 3 capas en el EDT)
  model/                     # MODELO (reglas del juego, sin dependencias de UI)
    - board/                 # tablero, celdas, posiciones, direcciones
    - box/                   # jugador y jerarquía de cajas (Template Method)
    - tile/                  # jerarquía de baldosas y efectos (Strategy)
    - effect/                # estrategias de efecto al entrar
    - mechanism/             # cerrojos y muros (Observer)
    - game/                  # Level, Game, estados, derrotas, puntaje, eventos
    - state/                 # patrón State
    - event/                 # patrón Observer (eventos)
    - loss/                  # patrón Strategy (derrotas)
    - score/                 # patrón Strategy (puntaje)
    - history/               # deshacer (Command + Memento)
  view/                      # VISTA Swing
  controller/                # CONTROLADOR teclado, HUD, reloj
  factory/                   # carga de niveles (Factory Method + Builder)
  port/                      # interfaces de salida (DIP)
  service/                   # adaptadores: sonido, persistencia
  exception/
```

3.2 Aislamiento MVC

- El aislamiento entre capas es estricto: Ninguna clase del paquete `model/` importa `javax.swing.*` ni `java.awt.*`.
- La vista se entera de los cambios del modelo exclusivamente a través del patrón Observer (`GameListener`). La vista nunca modifica el modelo: solo lee y repinta.
- Los controladores reciben los eventos de Swing (pulsaciones de teclado, clics en botones) y los traducen en llamadas a la fachada `Game`. Nunca acceden a `Board`, `Cell` o `UndoHistory` directamente.
- El único punto de cruce entre las tres capas es la clase `Main`, que arma el modelo, construye la vista, crea los controladores y los conecta dentro del Event Dispatch Thread (EDT) de Swing.

4. Patrones de diseño GoF aplicados

4.1 Factory Method (Creacional)

> *"Define una interfaz para crear un objeto, pero deja que las subclases decidan qué clase instanciar."* - *Gamma et al. (1994), p. 107.*

Problema: el archivo de texto del nivel usa caracteres (#, $, ~, P, etc.) para representar las celdas. La lógica de parseo no debería conocer cada tipo de baldosa ni de caja.

Solución:

Rol GoF	Clase
Creator (interfaz)	`CellCreator` (método `create(Position, char)`)
Concrete Creator	`WallCreator`, `DoorCreator`, `KeyTileCreator`, `PortalCreator`, `WeakBoxCreator`
Product	`CellContent` (record inmutable que agrupa Tile + Occupant)

En `LevelLoader`, un `Map<Character, CellCreator>` asocia cada carácter a su fábrica. Agregar un nuevo tipo de baldosa al juego consiste en implementar un `CellCreator` y registrarlo en el mapa, sin modificar el parser (OCP).

4.2 Builder (Creacional)

> *"Separa la construcción de un objeto complejo de su representación."* - *Gamma et al. (1994), p. 97.*

Problema: construir un `Level` requiere validar 5 reglas de consistencia complejas: balance de cajas vs. destinos por tipo, apareamiento de portales, correspondencia cerrojo/muro, cierre perimetral y jugador único.

Solución:

Rol GoF	Clase
Director	`LevelLoader` (lee el archivo de texto y dirige la construcción)
Builder	`LevelBuilder` (acumula las celdas con `withHeader()`, `addCell()`, `build()`)

La validación y el cableado bidireccional de portales y grupos cerrojo/muro ocurren en el paso final `build()`, garantizando que ninguna instancia inválida de Level llegue a existir. Si alguna regla falla, se lanza `InvalidLevelException` con el número de línea del archivo.

4.3 Template Method (Comportamiento)

> *"Define el esqueleto de un algoritmo en una operación, difiriendo algunos pasos a las subclases." - Gamma et al. (1994), p. 325.*

Problema: cada tipo de caja reacciona distinto al ser empujada (`WeakBox` se daña, `NormalBox` no), pero el esqueleto del empuje y del guardado es siempre el mismo.

Solución:

```
// Box.java (clase abstracta)
public final PushResult push() {
    return onPushed(); // hook
}
public final StorageResult storeAt(StoragePoint point) {
    return onStored(point); // hook
}

// WeakBox.java (redefine el hook)
@Override
protected PushResult onPushed() {
    if (remainingPushes == 0) return PushResult.BROKEN;
    remainingPushes--;
    return PushResult.PUSHED;
}

// KeyBox.java (redefine el hook)
@Override
protected StorageResult onStored(StoragePoint point) {
    return doorOpened ? StorageResult.STORED : StorageResult.KEY_NOT_USED;
}
```

Los métodos `push()` y `storeAt()` son final: las subclases solo redefinen los hooks. El tablero llama a `box.push()` sin conocer el tipo concreto de caja.

4.4 State (Comportamiento)

> *"Permite a un objeto alterar su comportamiento cuando su estado interno cambia. El objeto parecerá haber cambiado de clase."* - *Gamma et al. (1994), p. 305.*

Problema: el juego tiene 6 fases distintas (menú, jugando, nivel ganado, nivel perdido, deadlock con arreglo, juego completo). Cada fase tiene reglas distintas sobre qué teclas aceptar y qué transiciones permitir.

Solución:

Rol GoF	Clase
State (interfaz)	`GameState` (`acceptsInput()`, `onMove()`, `onUndo()`, `onNextLevel()`)
ConcreteState x6	`MenuState`, `PlayingState`, `WonState`, `LostState`, `BlockedState`, `GameCompletedState`
Context	`Game`

`Game` no tiene ni un solo if o switch sobre el estado. Delega cada acción al estado actual y reemplaza su estado por el que le devuelven:

```
// Game.java
public void move(Direction d) { state = state.onMove(this, d); }
public void undo()            { state = state.onUndo(this); }
public void nextLevel()       { state = state.onNextLevel(this); }
```

El bloqueo del teclado no es un booleano suelto: es `state.acceptsInput()`. Cada estado concreto solo escribe las transiciones que le corresponden.

4.5 Observer (Comportamiento) - Doble aplicación

> *"Define una dependencia uno-a-muchos entre objetos para que cuando un objeto cambie de estado, todos sus dependientes sean notificados automáticamente."* - *Gamma et al. (1994), p. 293.*

4.5.1 Eventos de partida

Problema: la vista, el sonido, la niebla de guerra y la persistencia de progreso necesitan reaccionar ante cada acción del juego, pero el modelo no puede conocer Swing ni la capa de servicios.

Rol GoF	Clase
Subject	`Level` (`addListener(GameListener)`, `publish(GameEvent)`)
Observer (interfaz)	`GameListener` (`onGameEvent(GameEvent)`)
ConcreteObserver	`BoardView` (repinta), `HudView` (actualiza contadores), `SoundService` (reproduce audio), `ProgressSaver` (persiste al ganar)
Mensaje	`GameEvent` (contiene EventType, Position y Direction)

4.5.2 Mecanismo cerrojo / puerta

Problema: cuando una caja llave cae sobre un casillero cerrojo, deben abrirse todos los muros de la misma letra. Pero `KeyTile` y `Door` no deben conocerse directamente.

Rol GoF	Clase
Subject	`KeyTile` (`addListener(KeyTileListener)`)
Observer (interfaz)	`KeyTileListener` (`onKeyTileActivated(char group)`)
ConcreteObserver	`DoorGroup` (agrupa todos los Door de una letra y los abre de una vez)

```
// KeyTile.java (notifica)
public LandingOutcome receiveBox(Box box) {
    if (!box.opensDoors()) return LandingOutcome.NONE;
    box.markDoorOpened();
    for (KeyTileListener listener : listeners) {
        listener.onKeyTileActivated(group);
    }
    return LandingOutcome.DOOR_OPENED;
}
```

4.6 Strategy (Comportamiento) - Triple aplicación

> *"Define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables."* - *Gamma et al. (1994), p. 315*.

4.6.1 Efectos de entrada al terreno

Rol GoF	Clase
Strategy	`EntryEffect` (`resolve(Board, Occupant, Position, Direction): Position`)
ConcreteStrategy x3	`NoEffect` (devuelve la posición tal cual), `SlideEffect` (hielo: avanza recursivamente), `TeleportEffect` (portal: posición de la salida)
Context	`Board` (aplica `tile.getEntryEffect().resolve(...)` tras cada entrada)

4.6.2 Condiciones de derrota

Rol GoF	Clase
Strategy	`LossCondition` (`isMet(Level): boolean`, `getReason(): LossReason`)
ConcreteStrategy x5	`BrokenBoxCondition`, `WrongStorageCondition`, `EarlyKeyStorageCondition`, `DeadlockCondition`, `TimeOutCondition`

4.6.3 Cálculo de puntaje

Rol GoF	Clase
Strategy / ConcreteStrategy	`ScoreCalculator` / `StandardScoreCalculator` (`calculate(...): ScoreResult`)

4.7 Memento (Comportamiento)

Problema: con hielo, portales, puertas que se abren irreversiblemente y casilleros cerrojo que se consumen, invertir un movimiento es irresoluble. Un solo empuje puede desencadenar múltiples alteraciones de estado.

Rol GoF	Clase
Originator	`Level` (`saveToMemento()`, `restoreFrom(LevelMemento)`)
Memento	`LevelMemento` (foto inmutable del tablero y estado interno de cajas/puertas)
Caretaker	`UndoHistory` (cola circular de 15 fotos, 5 pasos por uso, 3 usos consecutivos)

4.8 Command (Comportamiento)

Rol GoF	Clase
Command (interfaz)	`Command` (`execute()`)
ConcreteCommand	`MoveCommand` (encapsula la dirección y la referencia al Game)
Invoker	`InputController` (enlaza tecla direccional a `MoveCommand` en el `ActionMap` de Swing)

4.9 Flyweight (Estructural)

Solución: `SpriteCatalog` actúa como FlyweightFactory, utilizando un `Map<SpriteKey, BufferedImage>` para cargar cada PNG una sola vez. Las celdas de igual tipo comparten la referencia en memoria.

4.10 Facade / Fachada (Estructural)

Solución: `Game` es la única fachada del modelo hacia el controlador. `InputController` y `HudController` solo hablan con `Game`. `GameWindow` actúa como fachada de la vista Swing.

4.11 Repository / Adapter (Estructural + DIP)

Solución: El modelo depende de la abstracción `ProgressRepository`; la implementación concreta en disco (`FileProgressRepository`) vive en la capa de servicios. Esto respeta la Arquitectura Hexagonal / Puertos y Adaptadores.

4.12 Doble Despacho (Double Dispatch)

Problema: el tablero necesita saber si un ocupante puede entrar a una baldosa, pero la regla depende de ambos tipos (ocupante y baldosa).

Solución: El tablero llama `occupant.canEnter(tile)` sin `instanceof`. El `Player` le pregunta a la baldosa `tile.allowsPlayer()`, y la `Box` pregunta `tile.allowsBox(this)`.

5. Patrones GRASP aplicados

Patrón GRASP	Dónde se aplica	Evidencia
Information Expert	`Board`, `Box`, `Tile`, `LevelStats`	El tablero conoce todas las celdas - valida y ejecuta movimientos. La caja sabe si aguanta el empuje.
Creator	`LevelBuilder`	Crea `Cell[][]`, `Board` y `Level` porque agrega sus partes.

Patrón GRASP	Dónde se aplica	Evidencia
Controller	`InputController`, `HudController`	Reciben eventos de Swing y delegan en `Game`.
Low Coupling	Paquetes model / view / controller / port	El modelo no importa nada de UI.
High Cohesion	Un paquete por concepto	board, box, tile, history, loss, score, state tienen única razón de cambio.
Polymorphism	Jerarquía de `Tile` y `Box`	Evita switch sobre tipos en el motor de juego.
Pure Fabrication & Indirection	`LevelLoader`, `UndoHistory`, `DoorGroup`	Clases para bajar acoplamiento. KeyTile y Door no se ven directamente.
Protected Variations	`CellCreator`, `LossCondition`, `EntryEffect`	Puntos de variación aislados detrás de interfaces.

6. Principios SOLID

- **Single Responsibility (SRP):** Cada clase tiene una única responsabilidad. Ej: `UndoHistory` solo gestiona snapshots.
- **Open/Closed (OCP):** Abierto a extensión, cerrado a modificación. Se añaden tipos de baldosas o condiciones de derrota implementando interfaces, sin tocar clases base.
- **Liskov Substitution (LSP):** Subclases de `Tile` y `Box` son sustituibles sin violar invariantes.
- **Interface Segregation (ISP):** Interfaces mínimas. Ej: `Command` tiene 1 método, `Occupant` tiene 2.
- **Dependency Inversion (DIP):** `Game` depende de `IntFunction<Level>` (abstracción) y no de `LevelLoader`.

7. Diseño de niveles

Nivel	Nombre	Tamaño	Introduce	Empujes
01	Primeros pasos	7 × 8	Cajas normales + destinos (tutorial)	6
02	Carga frágil	8 × 10	Cajas frágiles con los empujes justos	10
03	Pista de hielo	9 × 11	Terreno resbaladizo para jugador y cajas	4
04	Bajo llave	9 × 11	Caja llave + casillero cerrojo + muro + niebla de guerra	11
05	El almacén final	9 × 13	Portales + los tres tipos de caja + muro	12

8. Interfaz gráfica (Swing)

Arquitectura de la vista: `GameWindow` (JFrame) gestiona un `CardLayout` entre `MenuView`, `BoardView` y `SummaryView`. `HudView` muestra contadores y botones en la parte superior del panel principal.

Dibujo del tablero: Celdas fijas de 48px, dibujadas en capas (baldosa → ocupante → niebla de guerra con AlphaComposite). Antialiasing NEAREST_NEIGHBOR para preservar pixel art.

Controladores y Sonido: `InputController` utiliza ActionMap/InputMap de Swing para resolver problemas de foco. `SoundService` implementa `GameListener` usando `javax.sound.sampled.Clip` precargado (WAV PCM 16-bit).

Assets: 21 sprites PNG de 48x48 px y 9 efectos de sonido WAV. Todos generados programáticamente y libres de derechos (CC0).

9. Suite de pruebas

108 tests JUnit 5 en 21 clases, verificando parseo, soluciones, mecánicas de juego, eventos, calculadores y persistencia. Todos pasan (BUILD SUCCESS).

10. Métricas del proyecto

Clases de producción (src/main)	89
Clases de prueba (src/test)	22
Líneas de producción	3.177
Líneas de test	1.719
Tests JUnit 5	108 (todos en verde)
Patrones GoF aplicados	12
Versión de Java	21
Framework gráfico	Swing (javax.swing.*)

11. Decisiones de diseño relevantes

- **Memento vs. Undo inverso:** Elegido Memento porque el hielo, portales y aperturas de puertas vuelven al undo inverso matemáticamente irresoluble o sumamente acoplado.
- **Destino vacío como agujero:** Bloquea al jugador, permitiendo caminar por encima solo cuando hay una caja guardada (hace de puente). Evita superposición de clases en la misma celda.
- **Consecutividad del deshacer:** La ventana de 15 movimientos se desplaza. Tras agotar 3 usos seguidos, hacer un nuevo movimiento reinicia el contador al liberar el estado.
- **InputMap vs. KeyListener:** Resuelve problemas donde interactuar con el HUD hacía perder el foco del teclado.

12. Uso de herramientas de IA

Se utilizaron modelos de lenguaje para apoyo en generación de código (especialmente Swing y generación de assets), análisis de estructura y validación de documentación. El autor tomó todas las decisiones de diseño y revisó la correcta aplicación de conceptos contra bibliografía de la materia.

13. Conclusiones

El proyecto demuestra la aplicación práctica de 12 patrones GoF, 9 patrones GRASP y 5 principios SOLID en un sistema real. Cada patrón responde a un problema (ej. State elimina ifs espagueti, Memento permite el deshacer complejo, Observers desacoplan la interfaz visual). El resultado es robusto, testable (108 tests exitosos) e íntegramente desvinculado de la interfaz de usuario.

Bibliografía

- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design* (3ra ed.). Prentice Hall.
- Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

14. Anexo - Diagramas de clases

(Las imágenes de los diagramas UML — A.1 Tablero y baldosas, A.2 Ciclo de juego y estados, A.3 Historial, carga y niveles, A.4 Vista, controlador y servicios — se encuentran en el directorio documentation/umls/ del repositorio principal.)